# Final Projects 2026/27

Speaker: Jeremy Miller (Shamoon College of Engineering (SCE))

Workshop

14th June 2026

# The succintness gap of Quantum Finite Automaton

1. **The succintness gap of Quantum Finite Automaton**
2. Turing machine model of DNA decryption
3. Trading Eye: Optimizing Stock-Market Trading Strategies
4. Quantum computing methods of improving time and space-complexity
5. A combinatoric model for genetic mutation
6. Quantum Entanglement methods for mapping key distributions
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A $DFA$ exists in only one state at a time.
- A $QFA$ exists in a **superposition of states**.
- i.e. a $QFA$ exists in all of its states at the same time.
- A non-deterministic $QFA$ can recognize non-regular languages.

# Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A $DFA$ exists in only one state at a time.
- A $QFA$ exists in a **superposition of states**.
- i.e. a $QFA$ exists in all of its states at the same time.
- A non-deterministic $QFA$ can recognize non-regular languages.

# Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A $DFA$ exists in only one state at a time.
- A $QFA$ exists in a **superposition of states**.
- i.e. a $QFA$ exists in all of its states at the same time.
- A non-deterministic $QFA$ can recognize non-regular languages.

# Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A $DFA$ exists in only one state at a time.
- A $QFA$ exists in a **superposition of states**.
- i.e. a $QFA$ exists in all of its states at the same time.
- A non-deterministic $QFA$ can recognize non-regular languages.

# Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A $DFA$ exists in only one state at a time.
- A $QFA$ exists in a **superposition of states**.
- i.e. a $QFA$ exists in all of its states at the same time.
- A non-deterministic $QFA$ can recognize non-regular languages.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a\in\Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle\psi_1|\psi_2\rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- A Measure-Once $QFA$ is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a\in\Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, $H$.
- A **Hilbert space** is a vector space with a defined inner-product $\langle\psi_1|\psi_2\rangle$ for all $\psi_1, \psi_2 \in H$.
- $\Sigma$ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in $H$ that defined the **initial state**.
- For every letter $a \in \Sigma$, $U_a$ is a **unitary matrix** that dictates how the $QFA$ calculates letters, analgous to the transition function of $DFA$.
- $F \in H$ is the set of **accepting states**.

# Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator $U_{\sigma_i}$ acts on each state vector to evolve the $QFA$ to the next state vector:

- **Acceptance:**The probablity that the $QFA$ accepts $w$ is the quared norm of the projection of the final state onto the accepting subspace:

# Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator $U_{\sigma_i}$ acts on each state vector to evolve the $QFA$ to the next state vector:

$$|\psi_w\rangle = U_{\sigma_n} U_{\sigma_{n-1}} \cdots U_{\sigma_1} |\psi_0\rangle \ .$$

- **Acceptance:** The probablity that the $QFA$ accepts $w$ is the quared norm of the projection of the final state onto the accepting subspace:

$$P\left(w \in L(M)\right) = \sum_{\psi \in F} \langle \psi | \psi \rangle \ .$$

# Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator $U_{\sigma_i}$ acts on each state vector to evolve the $QFA$ to the next state vector:

$$|\psi_w\rangle = U_{\sigma_n} U_{\sigma_{n-1}} \cdots U_{\sigma_1} |\psi_0\rangle \ .$$

- **Acceptance:**The probablity that the $QFA$ accepts $w$ is the quared norm of the projection of the final state onto the accepting subspace:

$$P\left(w \in L(M)\right) = \sum_{\psi \in F} \langle \psi | \psi \rangle \ .$$

# Problem 1

- **Problem 1:** For any language $L$ accepted by a DFA, $A$, does there always exist an equivalent QFA, $B$ that accepts $L$?
-

$$\forall\, A \in DFA:\ L = L(A) \qquad \exists\, B \in QFA:\ L = L(B) \qquad ?$$

$$L(D) = L(Q)\ ?$$

# Problem 1

- **Problem 1:** For any language $L$ accepted by a DFA, $A$, does there always exist an equivalent QFA, $B$ that accepts $L$?
-

$$\forall\, A \in DFA:\ L = L(A) \qquad \exists\, B \in QFA:\ L = L(B) \qquad ?$$

$$L(D) = L(Q)\ ?$$

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Problem 2

- Let $L$ be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let $A$ be the minimum $DFA$ that accepts $L$, and $B$ be the minimum $QFA$ that accepts $L$.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language $L$, calculate the minimal QFA, $B$ such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language $L$.

# Turing machine model of DNA decryption

1. ✓ The succintness gap of Quantum Finite Automaton
2. **Turing machine model of DNA decryption**
3. Trading Eye: Optimizing Stock-Market Trading Strategies
4. Quantum computing methods of improving time and space-complexity
5. A combinatoric model for genetic mutation
6. Quantum Entanglement methods for mapping key distributions
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid ($DNA$) is a double sequence of four different connected molecules:
  1. Adenine ($A$),
  2. Thymine ($T$),
  3. Cytosine ($C$),
  4. Guanine ($G$).
-

| A | T | G | G | C | T | A | C | G | T | A | A | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T | G | C | C |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence (12 Base Singlets)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence with Amino Acids**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Met | | | Ala | | | Thr | | | Stop | | |

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

# Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence (12 Base Singlets)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence with Amino Acids**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Met | | | Ala | | | Thr | | | Stop | | |

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

# Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence (12 Base Singlets)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence with Amino Acids**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Met | | | Ala | | | Thr | | | Stop | | |

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

# Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

| T | A | C | C | G | A | T | G | C | A | T | T |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence (12 Base Singlets)**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|

- **RNA Sequence with Amino Acids**

| A | T | G | G | C | T | A | C | G | T | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Met | | | Ala | | | Thr | | | Stop | | |

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

# Trading Eye: Optimizing Stock-Market Trading Strategies

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
3. **Trading Eye: Optimizing Stock-Market Trading Strategies**
4. Quantum computing methods of improving time and space-complexity
5. A combinatoric model for genetic mutation
6. Quantum Entanglement methods for mapping key distributions
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Trading Eye: Optimizing Stock-Market Trading Strategies

# Quantum computing methods of improving time and space-complexity

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
4. **Quantum computing methods of improving time and space-complexity**
5. A combinatoric model for genetic mutation
6. Quantum Entanglement methods for mapping key distributions
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Quantum computing methods of improving time and space-complexity

# A combinatoric model for genetic mutation

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity

5. **A combinatoric model for genetic mutation**
6. Quantum Entanglement methods for mapping key distributions
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# A combinatoric model for genetic mutation

# Quantum Entanglement methods for mapping key distributions

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation

6. **Quantum Entanglement methods for mapping key distributions**
7. Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Quantum Entanglement methods for mapping key distributions

# Shor's algorithm for decrypting RSA ciphertext in the absence of keys

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation
- ✓ Quantum Entanglement methods for mapping key distributions

7. **Shor's algorithm for decrypting RSA ciphertext in the absence of keys**
8. Stock market strategies based on the Black-Scholls equation and market reversal
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Shor's algorithm for decrypting RSA ciphertext in the absence of keys

# Stock market strategies based on the Black-Scholls equation and market reversal

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation
- ✓ Quantum Entanglement methods for mapping key distributions
- ✓ Shor's algorithm for decrypting RSA ciphertext in the absence of keys
8. **Stock market strategies based on the Black-Scholls equation and market reversal**
9. Combinatorial calculation of the probability of gene mutation
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Stock market strategies based on the Black-Scholls equation and market reversal

The goal of this project is to design robust algorithmic trading strategies by integrating the Black-Scholes model with statistical methods for detecting random market reversals. The Black-Scholes mathematical model estimates the theoretical value of put and call options based on dynamic variables like volatility and time to expiration. Combining this continuous-time finance model with stochastic calculus allows us to model random walks and construct delta neutral trading portfolios, successfully identifying probabilistic mean-reversion anomalies in asset prices for optimized trading.

# Combinatorial calculation of the probability of gene mutation

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation
- ✓ Quantum Entanglement methods for mapping key distributions
- ✓ Shor's algorithm for decrypting RSA ciphertext in the absence of keys
- ✓ Stock market strategies based on the Black-Scholls equation and market reversal

9. **Combinatorial calculation of the probability of gene mutation**
10. A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.
11. Building a Large Language Model to solve analytical equations.

# Combinatorial calculation of the probability of gene mutation

The goal of this project is to develop a computational model that calculates the probability of hereditary diseases based on complex gene combinations and specific mutations. Genetic predispositions are often polygenic, meaning multiple genes interact to influence a single phenotype or overall disease risk. By applying probabilistic models and combinatorial algorithms to genomic datasets, we can systematically map allele frequencies and interactions to accurately predict the statistical likelihood of specific genetic disorders.

# A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation
- ✓ Quantum Entanglement methods for mapping key distributions
- ✓ Shor's algorithm for decrypting RSA ciphertext in the absence of keys
- ✓ Stock market strategies based on the Black-Scholls equation and market reversal
- ✓ Combinatorial calculation of the probability of gene mutation

10. **A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.**
11. Building a Large Language Model to solve analytical equations.

# A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.

The goal of this project is to implement and optimize numerical algorithms for solving coupled systems of ordinary and partial differential equations while rigorously calculating their time and space complexity. Many physical and biological systems are modeled using continuous differential equations lacking closed-form solutions. By focusing on the Runge-Kutta method and other discretization techniques, we aim to approximate the dynamic behavior of these systems with high algorithmic precision. The project will specifically analyze the improved efficiency of optimized Runge-Kutta variations and benchmark their memory usage and execution time.

# Building a Large Language Model to solve analytical equations.

- ✓ The succintness gap of Quantum Finite Automaton
- ✓ Turing machine model of DNA decryption
- ✓ Trading Eye: Optimizing Stock-Market Trading Strategies
- ✓ Quantum computing methods of improving time and space-complexity
- ✓ A combinatoric model for genetic mutation
- ✓ Quantum Entanglement methods for mapping key distributions
- ✓ Shor's algorithm for decrypting RSA ciphertext in the absence of keys
- ✓ Stock market strategies based on the Black-Scholls equation and market reversal
- ✓ Combinatorial calculation of the probability of gene mutation
- ✓ A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.

11. **Building a Large Language Model to solve analytical equations.**

# Building a Large Language Model to solve analytical equations.

The goal of this project is to construct a scaled-down Large Language Model (LLM) to rigorously explore the linear algebra and deep mathematical foundations of modern artificial intelligence. By building neural networks from the ground up, the project will investigate non-linear transformations driven by the sigmoid model and other continuous activation functions. The architecture's underlying mathematics relies heavily on high-dimensional linear algebra, specifically focusing on complex matrix multiplications, matrix inversions for weight space analysis, and gradient descent optimization during backpropagation. This mathematical approach illuminates how dense vector embeddings and probabilistic token predictions fundamentally emerge from pure algebraic operations.